Detailed Class Notes: Computer Organization

Unit: Processor Organization, General Register Organization, Stack Organization, and Addressing Modes

---

1. Processor Organization

1.1 Definition

Processor organization refers to the internal architecture of the CPU — how the Arithmetic Logic Unit (ALU), registers, and control unit are interconnected to execute instructions and process data.

1.2 Core Components

1. **Arithmetic Logic Unit (ALU):** Performs arithmetic operations (addition, subtraction, multiplication) and logical operations (AND, OR, NOT, XOR, shifts).
2. **Registers:** High-speed storage locations inside the CPU used to hold operands, intermediate results, and control information.
3. **Control Unit (CU):** Generates control signals that coordinate the fetch-decode-execute cycle and direct data movement between registers, ALU, and memory.

1.3 Data Flow Model

A typical arithmetic micro-operation follows this flow:

```
Operand Register 1 --> ALU
Operand Register 2 --> ALU
ALU Result --> Destination Register
```

Example (Register Transfer Notation):

R3 ← R1 + R2

If R1 = 25 and R2 = 15: R3 = 25 + 15 = 40

1.4 Categories of Processor Organization

Processors are broadly classified based on how they reference operands:

Type	Description
Single Accumulator Organization	One accumulator register holds one operand and the result implicitly
General Register Organization	Multiple registers, operands explicitly specified in instructions
Stack Organization	Operands implicitly taken from the top of a stack (LIFO)

1.5 Application / Case Study

- **Single accumulator organization** was common in early 8-bit microprocessors (e.g., Intel 8085), where the accumulator register (A) held one operand for nearly every ALU operation.
- **Stack-based organization** is used in Hewlett-Packard's RPN (Reverse Polish Notation) calculators, and forms the basis of the Java Virtual Machine (JVM), which is a stack-based execution engine.
- **General register organization** is used in most modern RISC processors (ARM, MIPS), which have 16–32 general-purpose registers to minimize memory access latency.

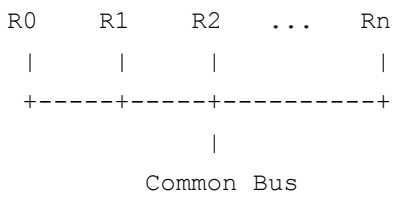
---

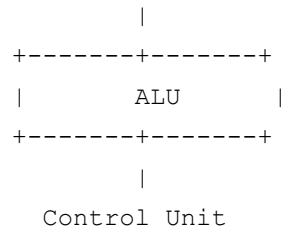
2. General Register Organization

2.1 Definition

In general register organization, the CPU contains a set of general-purpose registers (R0, R1, R2, ..., Rn) used for temporary storage of data during processing. Because these registers reside inside the CPU, access time is drastically shorter than accessing main memory (RAM).

2.2 Structural Diagram (Conceptual)





All registers connect to a common internal bus. The control unit selects:

- **SELA** – which register supplies input A to the ALU
- **SELB** – which register supplies input B to the ALU
- **OPR** – which operation the ALU performs
- **SELD** – which register receives the ALU's output

2.3 Advantages over Single Accumulator/Memory-based Organization

1. Reduces number of memory references, improving speed.
2. Allows more flexible instruction formats (e.g., three-address instructions: R1 ← R2 + R3).
3. Supports better compiler optimization (register allocation).

2.4 Mathematical Example

Three-address instruction: R1 ← R2 + R3

If R2 = 120, R3 = 80: R1 = 120 + 80 = 200

Multiply-Accumulate (MAC) operation, common in digital signal processing:

R0 ← R0 + (R1 × R2)

If R0 = 10, R1 = 4, R2 = 5: R0 = 10 + (4 × 5) = 10 + 20 = 30

2.5 Real-World Application / Case Study

- **Digital Signal Processors (DSPs)** used in mobile audio codecs, hearing aids, and voice-recognition chips rely heavily on register-based Multiply-Accumulate operations for tasks like Fast Fourier Transform (FFT) computation, filtering, and noise cancellation.
- **RISC architecture philosophy** (Reduced Instruction Set Computing) is built around the "load-store" model: only LOAD and STORE instructions access memory; all computation happens between registers. This is why ARM processors — used in virtually all smartphones — feature large general register files (16 in ARM32, 31 in ARM64/AArch64).

3. Stack Organization

3.1 Definition

A stack is a Last-In-First-Out (LIFO) storage structure. The last item inserted (pushed) into the stack is the first item removed (popped). In processor design, a stack can be implemented either as a dedicated set of registers (“register stack”) or as a reserved portion of main memory addressed via a Stack Pointer (SP).

3.2 Real-Life Analogy

A stack of cafeteria trays: trays are added to the top and removed from the top. The tray placed last is retrieved first.

3.3 Stack Operations

PUSH (insert an item onto the stack):

```
SP ← SP + 1
M[SP] ← DR
```

(SP = Stack Pointer, DR = Data Register holding the value to be pushed, M = Memory)

POP (remove the top item from the stack):

```
DR ← M[SP]
SP ← SP - 1
```

3.4 Numerical Example

Assume an empty stack, SP = 0:

Operation	SP after	Stack Contents (bottom → top)
PUSH 10	1	10
PUSH 25	2	10, 25
PUSH 7	3	10, 25, 7
POP → returns 7	2	10, 25
POP → returns 25	1	10

Note: 7 was pushed last, so it was popped first — confirming LIFO behavior.

3.5 Application: Postfix (Reverse Polish Notation) Expression Evaluation

One of the most important applications of a stack in processor design is evaluating arithmetic expressions without needing parentheses or operator precedence rules.

Infix expression: (3 + 4) × 5 **Postfix (RPN) equivalent:** 3 4 + 5 ×

Evaluation using a stack:

Step	Token	Action	Stack after action
1	3	Push 3	3
2	4	Push 4	3, 4
3	+	Pop 4, Pop 3, compute 3+4=7, Push 7	7
4	5	Push 5	7, 5
5	×	Pop 5, Pop 7, compute 7×5=35, Push 35	35

Result = 35

3.6 Real-World Case Studies

1. **Function Call Management (Call Stack):** Every time a program calls a function/procedure, the return address, parameters, and local variables are pushed onto the stack (an “activation record” or “stack frame”). On return, this frame is popped. Excessive recursion depth causes a **stack overflow**, a common runtime error.
2. **Java Virtual Machine (JVM):** JVM bytecode execution is entirely stack-based rather than register-based, simplifying portability across hardware architectures.
3. **Undo/Redo Functionality:** Applications like MS Word, Photoshop, and code editors implement “Undo” using a stack — the most recent action is the first to be undone.
4. **Browser Navigation History:** The “Back” button functionality is conceptually a stack-based operation.
5. **Compiler Design:** Compilers use stacks for syntax parsing (e.g., matching parentheses/brackets) and for converting infix expressions into postfix/prefix form.
6. **Calculators:** HP’s classic RPN calculators (e.g., HP-35, 1972) directly expose stack-based computation to the user.

4. Addressing Modes

4.1 Definition

An addressing mode specifies the rule for determining the **effective address (EA)** — the actual memory location or source of an operand — based on the address field(s) given in an instruction.

4.2 Real-Life Analogy

Consider different ways of specifying a destination when sending a letter:

- Give the exact house address (Direct)
- Say "the address is written on page 5 of my diary" (Indirect)
- Say "two streets from my house" (Relative)

4.3 Types of Addressing Modes

(a) Immediate Addressing Mode

The operand is specified directly within the instruction; no memory reference is needed.

Example: `MOV R1, #5` → R1 = 5

Formula: Operand = value in instruction

Use case: Loading constants, initializing counters/loop variables.

(b) Direct Addressing Mode

The address field of the instruction contains the actual address of the operand in memory.

Example: `MOV R1, [200]` → R1 = M[200]

Formula: EA = Address given in instruction

Use case: Accessing simple (non-array, non-pointer) variables.

(c) Indirect Addressing Mode

The address field of the instruction contains the address of a memory location that itself holds the address of the operand (a pointer).

Example: `MOV R1, [[200]]` If M[200] = 300, then R1 = M[300]

Formula: EA = M[Address given in instruction]

Use case: Implementing pointers, dynamic data structures (linked lists, trees).

(d) Register Addressing Mode

The operand is located in a CPU register rather than memory.

Example: `ADD R1, R2` → R1 = R1 + R2

Use case: Fastest possible operand access; used heavily in register-to-register (RISC-style) instructions.

(e) Register Indirect Addressing Mode

A register holds the address of the operand rather than the operand itself.

Example: `MOV R1, [R2]` → R1 = M[content of R2]

Formula: EA = content of register

Use case: Passing large data structures/arrays efficiently by reference, pointer-based programming (as in C).

(f) Indexed / Displacement Addressing Mode

The effective address is computed by adding a base register's content to a displacement value (often called an index).

Formula: EA = Base Register Content + Displacement

Example: If Base Register (BR) = 1000 and Displacement = 25: EA = 1000 + 25 = 1025

Application — Array Access: For array `A[]` with base address 1000 and element size 4 bytes, to access `A[5]`:

EA = Base Address + (Index × Element Size) EA = 1000 + (5 × 4) = 1020

Use case: Array traversal, record/structure field access in high-level languages.

(g) Relative Addressing Mode

The effective address is computed relative to the current value of the Program Counter (PC).

Formula: EA = PC + Address field of instruction

Use case: Branch and jump instructions; enables **position-independent code**, since the jump target is expressed relative to the current instruction rather than as an absolute address.

Real-world example: Shared libraries (.so/.dll files) rely on position-independent code so they can be loaded at different memory addresses across different program runs.

(h) Autoincrement / Autodecrement Addressing Mode

Similar to register indirect addressing, but the register's content is automatically incremented or decremented after (or before) the memory access.

Example (autoincrement): `MOV R1, [R2]+`

1. R1 ← M[R2]
2. R2 ← R2 + 1

Use case: Efficient sequential traversal of arrays, strings, and stacks without separate increment instructions — commonly used in string-copy and buffer-processing routines.

4.4 Summary Table

Addressing Mode	Effective Address Formula	Typical Use
Immediate	Operand is part of instruction	Constants
Direct	EA = Address in instruction	Simple variables
Indirect	EA = M[Address in instruction]	Pointers
Register	Operand in register	Fast computation
Register Indirect	EA = Content of register	Pointer via register
Indexed	EA = Base + Displacement	Arrays
Relative	EA = PC + Address	Branching, position-independent code
Autoincrement/decrement	EA = Register content (then updated)	Array/string traversal, loops

---

5. Key Takeaways for Exam/Revision

1. Processor organization defines how ALU, registers, and control unit interact; it is classified as accumulator-based, general-register-based, or stack-based.
2. General register organization minimizes memory access by keeping operands in fast internal registers — foundational to RISC design (ARM, MIPS).
3. Stack organization uses implicit LIFO addressing via a Stack Pointer; its major applications are postfix expression evaluation, function call management, and undo/redo systems.
4. Addressing modes define how the effective address of an operand is computed; indexed addressing is optimal for arrays, relative addressing for branching, and autoincrement/decrement for sequential data traversal.
5. Real processors combine multiple addressing modes and organizational styles — e.g., modern CPUs use general-register organization for computation but still maintain a hardware/software stack for function calls.

End of detailed notes.